MDA Audio

Investigación en el uso de gramáticas DSL para el análisis, diseño y generación de sintetizadores de audio software, siguiendo una arquitectura dirigida por modelos

Fernando Real Cisneros

Resumen

Contexto

Investigación sobre la aplicabilidad de arquitecturas dirigidas por modelos en el análisis, diseño y generación de código fuentes de sintetizadores de sonido software. Estructuración de proyectos de software orientados a la generación automática de aplicaciones que implementen sintetizadores de sonido.

Descripción del problema

Cómo estructurar, desde el análisis al diseño, un proyecto software que permita la generación automática de aplicaciones que implementen sintetizadores de sonido, en busca de ofrecer un mecanismo estandarizado tanto para la especificación y el diseño de los sintetizadores como para la generación de aplicaciones y su redefinición iterativa.

¿Por qué hay un problema?

La problemática del desarrollo software actual enfocado a la creación de sintetizadores de sonido se intenta abordar desde la creación de lenguajes visuales, no desde el punto de vista de la ingeniería software clásica, en la cual un análisis desemboca en un diseño, que a su vez termina en una implementación, y una iteración constante de redefición de todo ello. Lenguajes visuales tales como Pure Data o Max/MSP, permiten que músicos y *artistas sonoros* puedan crear sintetizadores de sonido de manera visual, pero no ofrecen garantías de que el software sea robusto o eficiente.

Resumen de la solución aportada

Siguiendo los principios MDA (Model Driven Architecture) se propone una solución basada en la creación de una serie de DSLs (Domain Specific Languages) que guien y acoten las dos principales fases postuladas por MDA, orientándolas hacia el desarrollo de sintetizadores de sonido en software: la especificación funcional del sintetizador (modelo CIM) y el diseño conceptual del mismo (modelo PIM). Estos modelos estrictos guiados por los DSL desarrollados, permiten acotar el problema, tanto en la generación automática del código fuente como en la redefinición iterativa de su análisis y diseño.

¿La solución aportada resuelve el problema?

La propuesta es mas ambiciosa de lo que este trabajo expone, que no es mas que la estructuración del análisis y diseño de este tipo de software de cara a su generación en

código. El problema mas amplio sería la estandarización del desarrollo de sintetizadores de sonido desde el prisma de arquitecturas dirigidas por modelos, de lo cual esta investigación pretende dar una opción viable.

Índice general

1. Introducción **5**

2. El problema **7**

2.1. Descripción del problema planteado 7

2.2. Opciones ya existentes para su resolución y limitaciones de las mismas . . . 8

3. Descripción detallada de la solución propuesta **9**

3.1. Descripción de alto nivel 9

3.2. Descripción funcional 9

3.3. Detalles del prototipo realizado 9

3.4. Casos de uso aplicados para las pruebas de demostración 9

4. Evaluación y comparación de la solución **10**

5. Como evaluar la solución **11**

6. Conclusiones **12**

A. Anexo A: Introducción a la Síntesis de Sonido Digital **13**

A.1. Fundamentos Básicos del Sonido 13

A.2. Componentes Principales de un Sintetizador 14

A.2.1. El Oscilador (Oscillator / VCO) 15

A.2.2. El Filtro (Filter / VCF) 16

A.2.3. El Amplificador (Amplifier / VCA) 16

A.2.4. Generador de Envolvente (Envelope / ADSR) 16

A.2.5. Oscilador de Baja Frecuencia (LFO) 17

A.2.6. El Mezclador (Mixer) 18
A.2.7. Imagen Espacial: Estéreo y Paneo (Pan) 18
A.2.8. Procesadores de Efectos 19
A.3. Lenguajes para el Desarrollo de Sintetizadores 21

B. Anexo B: El Lenguaje Pure Data (Pd) y su Complejidad Estructural 23
B.1. Programación Visual en Pure Data 23
B.1.1. Estructuración Modular: Subpatches y Abstracciones 24
B.2. Entorno Recomendado: Pd-l2ork (Purr Data) 25
B.2.1. Instalación y Configuración Básica 25
B.3. Estructura Interna del Archivo `.pd` 26
B.4. Desafíos para la Generación Automática 27

Índice de figuras 28

Capítulo 1

Introducción

El presente proyecto de investigación tiene como finalidad principal la estructuración del desarrollo de sintetizadores de sonido mediante la aplicación de metodologías *Model-Driven Architecture* (MDA). Para lograrlo, el sistema se fundamenta en las dos capas principales de abstracción que propone MDA: el *Computation Independent Model* (CIM) y el *Platform Independent Model* (PIM).

Con el objetivo de orientar y formalizar estos desarrollos, se han diseñado e implementado múltiples lenguajes de dominio específico o *Domain-Specific Languages* (DSL). Estos lenguajes permiten capturar la semántica y la arquitectura de los sintetizadores en diferentes niveles de abstracción.

Una de las grandes ventajas de esta estructuración basada en modelos estrictos es la capacidad de generar *prompts* altamente estructurados y fáciles de procesar por inteligencias artificiales. Esta integración brinda la posibilidad de iterar de manera continua y eficiente entre las fases de diseño y análisis de los modelos de síntesis de audio, explorando con ello nuevas posibles generaciones del código fuente objetivo.

Para materializar este enfoque conceptual, se ha desarrollado una herramienta integral. Esta plataforma ofrece un entorno completo para la definición visual de los modelos, facilitando el ciclo de vida del desarrollo. Los principales puntos que la herramienta ofrece al usuario final incluyen:

- **Gestión de modelos CIM y PIM:** Entornos dedicados para la abstracción del dominio de síntesis y la arquitectura técnica del sintetizador. Desarrollo gráfico de los modelos PIM, facilitando su iomplementación abstrayendonos de los aspectos de bajo nivel postulados por los DSL desarrollados.
- **Lenguajes DSL especializados:** Integración de gramáticas propias que validan la coherencia estructural de las conexiones y flujos de señales que debe tener una aplicación software de estas características.
- **Exportación asistida por IA:** Capacidad de exportar el contexto del proyecto

junto con los manuales de los DSL, adaptándolos de cara a generar un prompt estructurado y coherente, orientado a guiar a una IA en la generación de código funcional.

- **Validación continua:** Verificación en tiempo real para garantizar que los modelos cumplen con las restricciones del dominio antes de su implementación.

En conclusión, los DSL desarrollados constituyen el núcleo de este proyecto de investigación, intentando buscar con ellos una forma de estructurar desarrollos en un ambito muy concreto, como es éste de la creación de software especifico para audio digital. A su vez, la herrameinta desarrollada permite facilitar el desarrollo de estos modelos, proporcionando lo necesario para la generación automática del código fuente objetivo.

Este campo, la creación de software especializado en audio regido por modelos de abstracción, aún es un campo por explorar y que presenta muchas posibilidades de desarrollo, por lo que creo que esta investigación puede ser de utilidad para futuros proyectos en esta área.

Recomendación

La sistesis de audio digital y el desarrollo de software derivado, es una temática muy específica, sobre la cual el evaluador de éste trabajo no tendría porqué conocer nada a priori. Por ello, se han desarrollado dos anexos introductorios al respecto, los cuales podrían ser de utilidad para el lector que no esté familiarizado con la materia. El primero de ellos expone los aspectos fundamentales de la teoría de audio y síntesis de sonido digital (Anexo A), y el segundo los aspectos fundamentales del lenguaje de programación que se utiliza como ejemplo, en conjunto con JavaScript, para las generaciones de código fuente (Anexo B).

Capítulo 2

El problema

2.1. Descripción del problema planteado

El desarrollo de software orientado a la creación de sintetizadores de sonido constituye un dominio altamente especializado que, con frecuencia, se desvía de los paradigmas clásicos de programación. Históricamente, este campo ha carecido de metodologías formales que estructuren el ciclo de vida del desarrollo, priorizando una aproximación directa a la implementación (*bottom-up*) sobre un proceso sistemático de ingeniería. En esto ha pesado mucho el usuario al que los lenguajes desarrollados se dirigen, músicos, ingenieros de sonido, diseñadores de sonido, etc., los cuales no suelen tener conocimientos de programación. Salvo en los casos de lenguajes de propósito general como librerías adaptadas a la síntesis musical en JavaScript o C, u otras enfocadas a la investigación sonora como SuperCollider o Csound, la mayoría de las herramientas disponibles, con éxito de uso, para el desarrollo de sintetizadores de audio son framework visuales, como Pure Data o Max/MSP.

El problema fundamental radica en la ausencia de un flujo de trabajo que permita asentar un análisis sólido, el cual deba desembocar en un diseño arquitectónico que sirva como pilar para la generación del código fuente. Esta carencia metodológica impide la trazabilidad y la iteración efectiva entre las distintas fases del desarrollo. Sin un modelo formal intermedio, resulta extremadamente complejo volver atrás para redefinir el análisis o aplicar cambios en el diseño de forma que se reflejen de manera coherente y automática en el producto final. Esta desconexión entre la intención creativa y la implementación técnica limita la escalabilidad de los proyectos y dificulta la reutilización de componentes lógicos entre diferentes desarrollos.

2.2. Opciones ya existentes para su resolución y limitaciones de las mismas

Si bien es cierto que es posible utilizar lenguajes de propósito general (como C++ o Rust) para el desarrollo de sintetizadores, lo que permitiría teóricamente aplicar técnicas como las postuladas por *Model-Driven Architecture* (MDA) y ciclos de vida de ingeniería de software tradicionales, este no es un enfoque realista en la práctica. La realidad del sector muestra que los perfiles encargados de la creación de estas herramientas —músicos, artistas sonoros, diseñadores de audio e ingenieros de sonido— no suelen poseer una formación profunda en arquitectura de software o paradigmas de programación complejos.

Para estos usuarios, el uso de lenguajes de bajo nivel o marcos de trabajo genéricos supone una barrera de entrada técnica insalvable que asfixia la creatividad. Por ello, la industria ha convergido hacia herramientas de programación visual, que aunque facilitan la implementación inmediata, carecen habitualmente de las capas de abstracción necesarias para realizar un análisis y diseño formal previo e independiente de la implementación final.

Por todo esto, y tras buscar bastante información al respecto, no he encontrado nada que intente solventar este problema. Lo cual, por un lado hace pensar que quizás sea por la poca utilidad práctica final que pudiera tener este enfoque planteado, pero por otro lo hace interesante para su investigación

Capítulo 3

Descripción detallada de la solución propuesta

3.1. Descripción de alto nivel

3.2. Descripción funcional

3.3. Detalles del prototipo realizado

3.4. Casos de uso aplicados para las pruebas de demostración

Capítulo 4

Evaluación y comparación de la solución

Capítulo 5

Como evaluar la solución

Capítulo 6

Conclusiones

Apéndice A

Anexo A: Introducción a la Síntesis de Sonido Digital

Este anexo está diseñado como un tutorial introductorio para aquellos lectores que no posean conocimientos previos sobre la síntesis de sonido digital. A lo largo de estas páginas se abordarán los conceptos fundamentales de la teoría del sonido, los componentes básicos de un sintetizador y, finalmente, se presentarán algunos de los lenguajes de programación más utilizados en la actualidad para la implementación de estos sistemas.

A.1. Fundamentos Básicos del Sonido

El sonido, desde una perspectiva física, no es más que una variación de la presión del aire que se propaga en forma de ondas. Cuando estas ondas llegan a nuestro oído, el cerebro las interpreta como sonido. Para entender cómo funciona un sintetizador, primero debemos comprender dos propiedades fundamentales de estas ondas: la amplitud y la frecuencia.

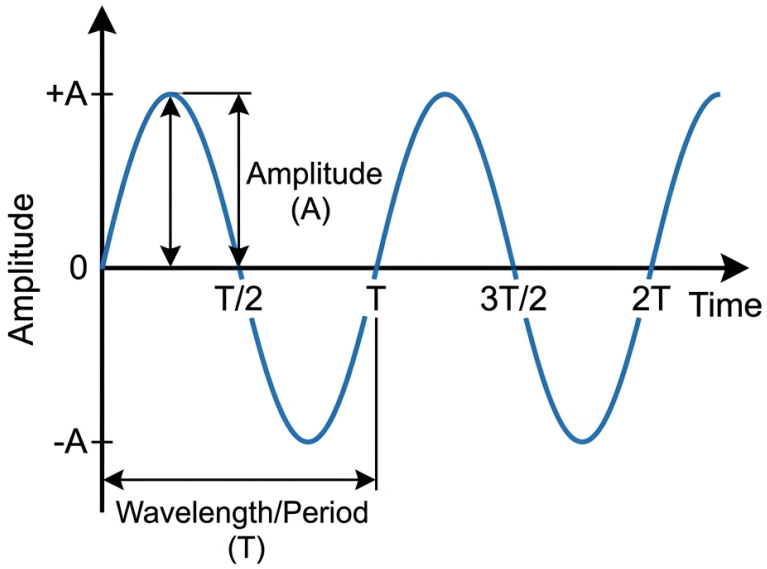

Figura A.1: Representación de una onda sonora básica ilustrando amplitud y frecuencia.

- **Amplitud:** Representa la altura de la onda. En términos perceptuales, la amplitud determina el *volumen* o intensidad del sonido. A mayor amplitud, el sonido percibido es más fuerte.
- **Frecuencia:** Es la velocidad a la que la onda vibra, es decir, el número de ciclos que completa en un segundo. Se mide en Hercios (Hz). La frecuencia determina el *tono* (altura) del sonido; las frecuencias altas producen sonidos agudos, mientras que las frecuencias bajas generan sonidos graves.

En el ámbito digital, el sonido continuo (analógico) se convierte en una serie de valores numéricos discretos mediante un proceso conocido como muestreo (*sampling*). La síntesis digital de sonido es, por tanto, el proceso de generar estos números mediante algoritmos matemáticos en tiempo real para ser enviados a un altavoz.

A.2. Componentes Principales de un Sintetizador

La mayoría de los sintetizadores modernos, ya sean analógicos o digitales, comparten una arquitectura similar basada en módulos interconectados. El paradigma más común es la *síntesis sustractiva*, donde se genera un sonido armónicamente rico y luego se le sustraen frecuencias para esculpir el timbre final.

En el contexto de esta investigación, esta arquitectura modular se corresponde directamente con el concepto de **Máquina** definido en nuestro *Computation Independent Model* (CIM). El CIM permite abstraer estas configuraciones complejas en unidades funcionales de alto nivel, facilitando la comprensión del flujo sonoro antes de entrar en detalles de implementación técnica.

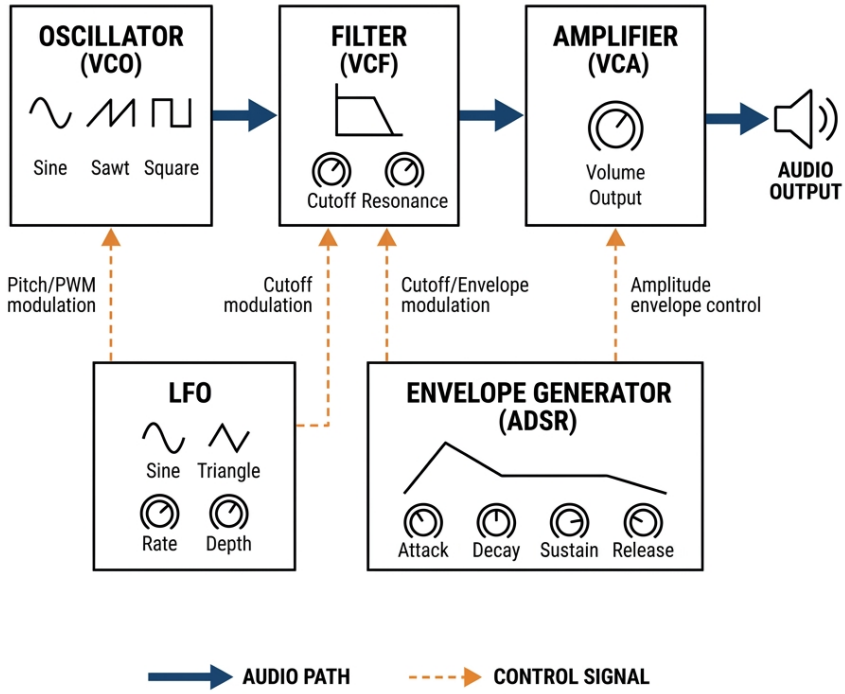

Figura A.2: Arquitectura en bloques de un sintetizador sustractivo clásico.

A continuación, detallamos los componentes esenciales. Cabe destacar que, en nuestro *Platform Independent Model* (PIM), cada uno de estos componentes se representa mediante un **Nodo**, cuyas propiedades técnicas (como la frecuencia o el tipo de onda) son configurables a través de los parámetros definidos en el DSL.

A.2.1. El Oscilador (Oscillator / VCO)

El oscilador es el corazón del sintetizador; es la fuente original del sonido. Su función es generar una forma de onda continua a una frecuencia específica (determinada, por ejemplo, por la tecla pulsada en un teclado). Las formas de onda más comunes son:

- **Senoidal (Sine):** Un sonido puro, sin armónicos adicionales, similar a un silbido suave.
- **Cuadrada (Square):** Un sonido hueco y penetrante, muy utilizado en la música de videojuegos clásicos (chiptune).
- **Diente de sierra (Sawtooth):** Un sonido brillante y rico en armónicos, ideal para cuerdas o metales sintetizados y bajos potentes.

A.2.2. El Filtro (Filter / VCF)

El filtro toma el sonido crudo del oscilador y altera su timbre eliminando (o atenuando) ciertas frecuencias. El filtro más común es el de *Paso Bajo* (Low Pass Filter), que permite pasar las frecuencias graves y recorta las agudas, haciendo que el sonido sea más oscuro o apagado. El parámetro *Cutoff* (frecuencia de corte) determina el punto exacto donde el filtro empieza a actuar.

A.2.3. El Amplificador (Amplifier / VCA)

El amplificador controla el volumen general de la señal de audio antes de ser enviada a la salida principal. Si el oscilador genera sonido continuamente, el amplificador actúa como una puerta que se abre y se cierra para permitir que escuchemos el sonido solo cuando deseamos (por ejemplo, al pulsar una tecla).

A.2.4. Generador de Envolvente (Envelope / ADSR)

Para que un sonido se sienta natural, su volumen (u otros parámetros) debe cambiar con el tiempo. Un piano suena fuerte al golpear la tecla y se desvanece gradualmente. El Generador de Envolvente, comúnmente del tipo ADSR, permite modelar este comportamiento temporal.

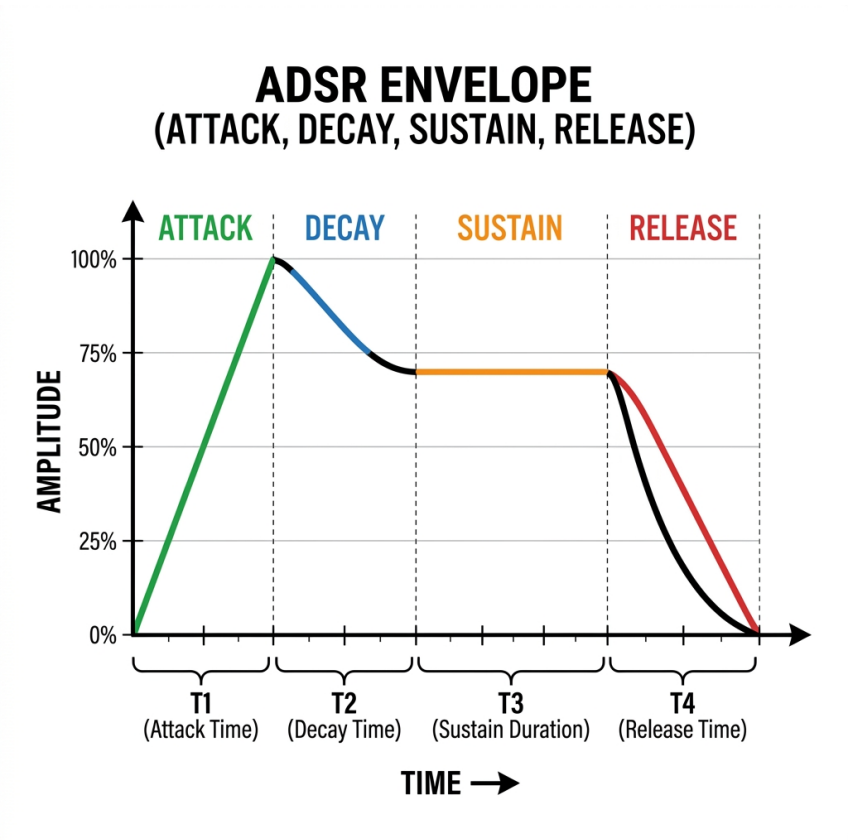

Figura A.3: Curva de un generador de envolvente ADSR.

Los cuatro parámetros del ADSR son:

- **Ataque (*Attack*):** El tiempo que tarda el sonido en alcanzar su volumen máximo desde que se inicia.
- **Decaimiento (*Decay*):** El tiempo que tarda en bajar desde el nivel máximo hasta el nivel de sostenimiento.
- **Sostenimiento (*Sustain*):** El nivel de volumen que se mantiene mientras se sigue pulsando la nota.
- **Relajación (*Release*):** El tiempo que tarda el sonido en desvanecerse hasta el silencio absoluto una vez que se suelta la nota.

A.2.5. Oscilador de Baja Frecuencia (LFO)

A diferencia del oscilador principal, el LFO (*Low Frequency Oscillator*) vibra a una velocidad tan lenta (generalmente por debajo de los 20 Hz) que no podemos escucharlo como un tono. En su lugar, se utiliza para automatizar de forma cíclica otros parámetros. Por ejemplo, si conectamos un LFO al volumen, obtendremos un efecto de *trémolo*; si lo conectamos al tono del oscilador, obtendremos un *vibrato*.

A.2.6. El Mezclador (Mixer)

En arquitecturas complejas con múltiples osciladores o fuentes de sonido (como ruido o *samples* pregrabados), el Mezclador es el componente encargado de sumar todas estas señales en un único bus de audio. Permite ajustar el balance de volumen relativo entre las diferentes fuentes antes de que pasen a la etapa de filtrado o amplificación.

Desde el punto de vista del modelado PIM, el flujo de señales entre osciladores, filtros y mezcladores se define mediante **Aristas** (o conexiones). Estas aristas no solo representan la ruta del audio, sino que en nuestro sistema validan la compatibilidad entre los puertos de salida y entrada de cada nodo, asegurando que el diseño del sintetizador sea técnicamente viable.

A.2.7. Imagen Espacial: Estéreo y Paneo (Pan)

En el diseño de sonido, no solo importa el timbre y el volumen, sino también la posición espacial del sonido.

- **Mono vs. Estéreo:** Una señal *mono* (monofónica) contiene un único canal de audio, lo que significa que suena igual en el altavoz izquierdo y en el derecho, percibiéndose en el centro geométrico del oyente. Una señal *estéreo* consta de dos canales independientes (izquierdo y derecho), permitiendo crear amplitud y movimiento espacial.
- **Paneo (*Panning*):** El control de paneo determina cómo se distribuye una señal mono entre los canales izquierdo y derecho de un sistema estéreo. Al ajustar el paneo, podemos posicionar el sonido en cualquier punto del panorama estéreo (por ejemplo, totalmente a la izquierda, al centro o moviéndose dinámicamente de un lado a otro).

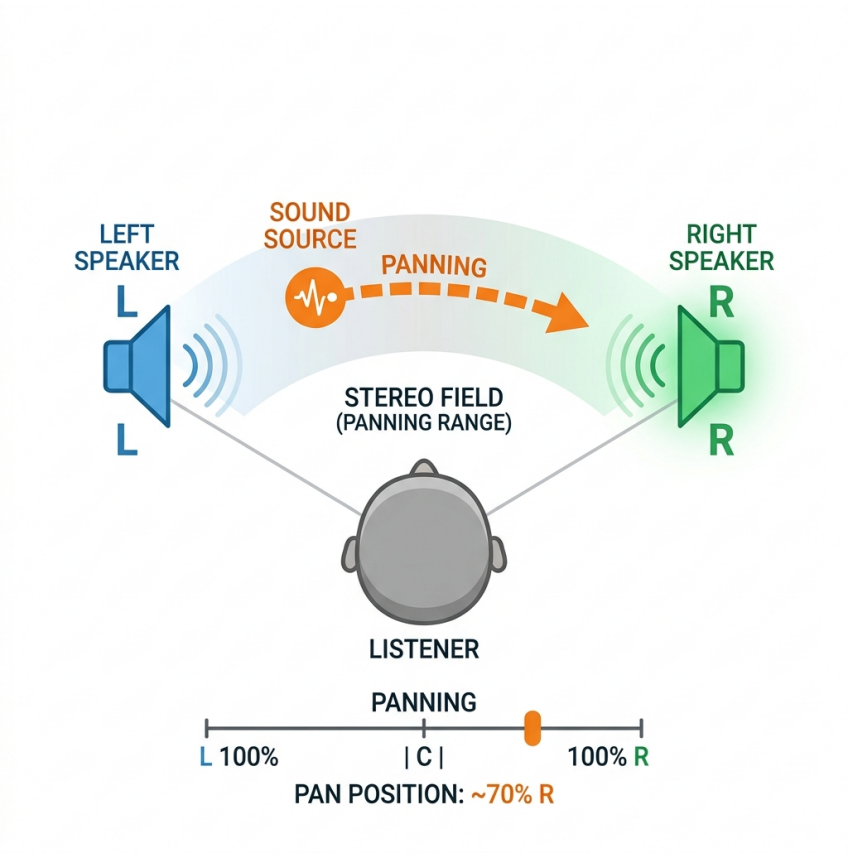

Figura A.4: Representación del campo estéreo y el efecto de paneo.

A.2.8. Procesadores de Efectos

Una vez que el sonido básico ha sido generado y esculpido por osciladores, filtros y envolventes, suele pasar por una cadena de efectos para añadir profundidad, carácter y textura. Los más comunes incluyen:

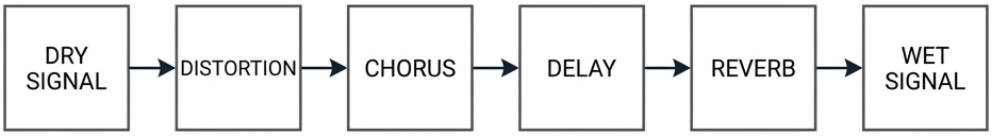

Figura A.5: Ejemplo de una cadena de señal (*Signal Chain*) con múltiples procesadores de efectos.

- **Reverberación (*Reverb*):** Simula el comportamiento acústico de un espacio físico (como una habitación, una sala de conciertos o una catedral). Añade una serie densa de ecos muy rápidos que difuminan el sonido, dándole profundidad y sensación de espacio.

- **Retardo (*Delay*):** Captura la señal de audio y la repite después de un tiempo determinado (tiempo de *delay*), creando un efecto de eco rítmico. Suelen tener un control de retroalimentación (*feedback*) para determinar cuántas veces se repite el eco.

- **Distorsión (*Distortion*):** Altera la forma de onda original *recortando* sus picos (clipping) o introduciendo armónicos adicionales. Genera un sonido más agresivo, cálido o áspero, fundamental en guitarras eléctricas y en muchos géneros de música electrónica.

- **Chorus y Flanger:** Ambos son efectos de modulación temporal. El *Chorus* duplica la señal y retrasa ligeramente la copia modulando su tono sutilmente, creando la ilusión de múltiples voces sonando al unísono. El *Flanger* hace algo similar pero con tiempos de retraso mucho más cortos, provocando un efecto de barrido armónico característico similar al sonido de un avión a reacción.

- **Compresor (*Compressor*):** Es un procesador de dinámica. Reduce el volumen de los picos más fuertes de la señal (cuando superan un cierto umbral) y permite subir el volumen general. Esto resulta en un sonido más *denso* y controlado, asegurando que ninguna parte suene demasiado fuerte ni demasiado floja.

- **Ecualizador (*Equalizer / EQ*):** Es un tipo de filtro altamente preciso que permite aumentar o atenuar rangos de frecuencia muy específicos sin afectar al resto del espectro, ideal para esculpir el sonido final y hacer que encaje bien en una mezcla musical.

A.3. Lenguajes para el Desarrollo de Sintetizadores

A medida que la síntesis de sonido se ha trasladado al entorno del *software*, han surgido múltiples lenguajes y entornos de programación especializados en el procesamiento de señales de audio digitales (DSP). A continuación se destacan los más relevantes en la industria y el ámbito académico:

- **Pure Data (Pd):** Es un lenguaje de programación visual de código abierto. En lugar de escribir líneas de código, el usuario conecta *cajas* (nodos) en la pantalla mediante *cables* (aristas) para crear el flujo de la señal. Es altamente popular en entornos interactivos, instalaciones artísticas y el desarrollo rápido de prototipos debido a su enfoque intuitivo.

- **Max/MSP:** Es el hermano comercial de Pure Data. Desarrollado por la empresa Cycling '74 (ahora parte de Ableton), ofrece una interfaz gráfica mucho más pulida, integración profunda con la popular estación de trabajo de audio digital *Ableton Live* a través de *Max for Live*, y una extensa biblioteca de objetos y extensiones comunitarias.

- **SuperCollider:** A diferencia de Pd y Max, SuperCollider es un lenguaje basado en texto. Está compuesto por un servidor de audio de alto rendimiento y un lenguaje de programación orientado a objetos. Es extraordinariamente potente y eficiente, siendo el lenguaje predilecto para la síntesis algorítmica, composición generativa y actuaciones de *live coding*.

- **CSound:** Uno de los lenguajes de programación de audio más antiguos y respetados, desarrollado originalmente en el MIT. Se basa en archivos de texto divididos

en una *orquesta* (que define los instrumentos) y una *partitura* (que indica cuándo tocarlos). Ofrece una fidelidad de audio excepcional y una inmensa biblioteca de *opcodes* (funciones).

- **Web Audio API:** No es un lenguaje independiente, sino una interfaz de programación (API) de alto nivel incorporada en los navegadores web modernos (mediante JavaScript). Permite la síntesis de audio, el enrutamiento complejo y el procesamiento de señales directamente en el navegador sin necesidad de *plugins* adicionales, democratizando el acceso a la creación de audio digital.

El dominio de estos entornos requiere una curva de aprendizaje considerable. Es precisamente aquí donde propuestas arquitectónicas de alto nivel, como las metodologías de *Model-Driven Architecture* y los DSL (como se expone en este proyecto), aportan valor al simplificar la transición desde la idea abstracta de un sintetizador hasta su implementación en estos lenguajes específicos.

Apéndice B

Anexo B: El Lenguaje Pure Data (Pd) y su Complejidad Estructural

Pure Data (Pd) es un entorno de programación visual de código abierto desarrollado originalmente por Miller Puckette en los años 90. Es ampliamente utilizado por músicos e investigadores para la creación interactiva de música por ordenador y obras multimedia. A diferencia de los lenguajes de programación tradicionales basados en texto, Pd permite a los usuarios desarrollar algoritmos conectando objetos gráficos en un *lienzo* o, como Pure Data los denomina, *patches*.

B.1. Programación Visual en Pure Data

En Pd, un programa se construye instanciando diferentes cajas u *objetos* y conectándolos mediante *cables*. Cada objeto realiza una función específica, como generar una onda matemática, sumar dos números o aplicar un filtro a una señal de audio.

El flujo de información viaja a través de las conexiones de arriba hacia abajo. Existen dos tipos principales de conexiones:

- **Cables de control:** Transmiten mensajes y números discretos, ideales para enviar eventos, notas musicales o modificar parámetros en tiempo real.
- **Cables de señal (*audio*):** Transmiten flujo continuo de audio a la frecuencia de muestreo del sistema (típicamente 44100 muestras por segundo). Los objetos que procesan estas señales se denotan con una tilde (`~`) al final de su nombre (por ejemplo, `osc~` o `dac~`).

A continuación se muestra un ejemplo visual de un *patch* básico en Pure Data:

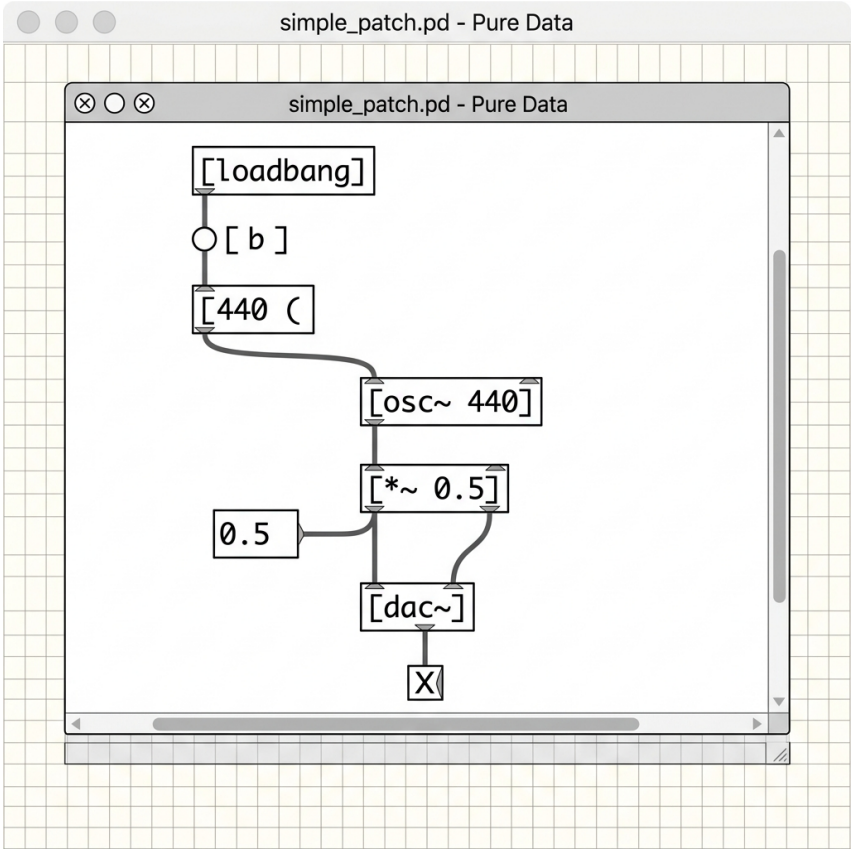

Figura B.1: Interfaz visual de un *patch* en Pure Data.

B.1.1. Estructuración Modular: Subpatches y Abstracciones

Al igual que en la ingeniería de software tradicional se utilizan funciones, métodos o clases para organizar y reutilizar el código, en Pure Data se emplea el concepto de *patch* (el archivo principal o lienzo de trabajo) y sus encapsulaciones.

A medida que un sintetizador crece en complejidad, el lienzo visual puede volverse inmanejable. Para solucionar esto, Pd permite la creación de sub-módulos mediante dos mecanismos principales:

- **Subpatches:** Se crean usando el objeto `[pd nombre_del_subpatch]`. Al hacer doble clic sobre él, se abre una nueva ventana en blanco donde se puede seguir conectando objetos. Es útil para ocultar el desorden visual y agrupar lógica relacionada dentro del mismo archivo.

- **Abstracciones:** Consisten en guardar un *patch* en un archivo `.pd` independiente y luego invocarlo desde otro *patch* simplemente escribiendo su nombre en una caja de objeto. Esto permite la verdadera reutilización de código; por ejemplo, se puede diseñar un oscilador complejo y reutilizarlo múltiples veces instanciando su abstracción.

Desde la perspectiva de las herramientas de generación automática desarrolladas en este proyecto, este concepto de encapsulación es fundamental. Un sub-módulo en Pure Data equivale directamente al concepto arquitectónico de ***Máquina*** definido en nuestros lenguajes DSL. Del mismo modo, esta misma equivalencia semántica se mapea a **clases** o **métodos** cuando el código objetivo es un lenguaje de texto estructurado como JavaScript u otros lenguajes soportados.

B.2. Entorno Recomendado: Pd-l2ork (Purr Data)

Aunque existe una versión original o *vanilla* de Pure Data mantenida por Miller Puckette, para este proyecto y para el desarrollo moderno en general, se recomienda encarecidamente el uso de **Pd-l2ork** (también conocido en algunas de sus distribuciones como Purr Data).

Esta bifurcación (*fork*) del proyecto original ofrece una interfaz gráfica basada en tecnologías web mucho más amigable, mejoras significativas de usabilidad y un vasto conjunto de librerías y extensiones preinstaladas que facilitan la creación de sintetizadores sin necesidad de compilar dependencias de terceros.

B.2.1. Instalación y Configuración Básica

Pd-l2ork es multiplataforma y puede descargarse de forma gratuita desde su sitio web oficial (`https://l2ork.bukvic.net/main/make-your-own-l2ork/software/`). La instalación sigue el proceso estándar de cualquier aplicación según el sistema operativo (Windows, macOS o Linux).

Un detalle crucial y un error extremadamente común entre los principiantes, es intentar hacer sonar un *patch* sin haber activado el motor de audio del programa. Para que cualquier objeto relacionado con la síntesis de sonido (aquellos terminados en `~`) genere salida acústica, es estrictamente necesario activar el procesamiento digital de señales.

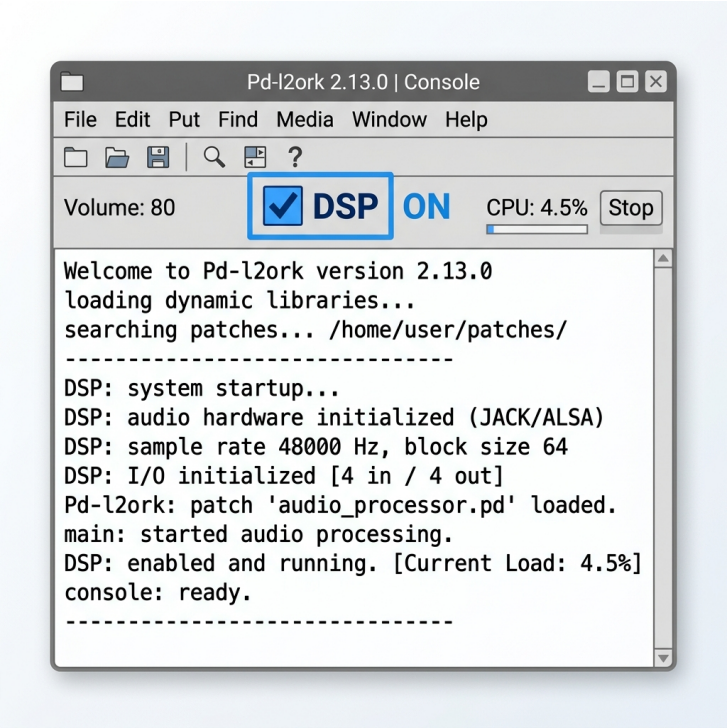

Figura B.2: Consola principal de Pd-l2ork destacando la casilla DSP.

Para ello, basta con abrir la ventana principal (Consola) de Pd-l2ork y asegurarse de que la casilla etiquetada como **DSP** (*Digital Signal Processing*) esté marcada, tal y como se observa en la Figura B.2. Sin este paso, el *patch* calculará datos de control, pero no emitirá ningún sonido.

B.3. Estructura Interna del Archivo `.pd`

A pesar de su interfaz visual e intuitiva, internamente Pure Data guarda estos *patches* en un formato de texto plano (con extensión `.pd`). La sintaxis de este archivo es muy particular y está optimizada para ser leída rápidamente por el motor de Pd, no por un humano.

El siguiente es un ejemplo de cómo se representa internamente un *patch* sencillo:

```
#N canvas 50 50 450 300 10;
#X obj 100 100 osc~ 440;
#X obj 100 150 *~ 0.5;
#X obj 100 200 dac~;
#X connect 0 0 1 0;
#X connect 1 0 2 0;
#X connect 1 0 2 1;
```

Código B.1: Contenido en texto plano de un archivo `.pd`.

Como se observa, el formato define:

- Definiciones del *canvas* (coordenadas y tamaño de la ventana).
- Declaración de objetos (`#X obj`) junto a sus coordenadas absolutas (X, Y) y sus parámetros de inicialización.
- Declaraciones de conexión (`#X connect`), las cuales no hacen referencia a los identificadores o nombres de los objetos, sino a sus **índices de creación relativos** en el archivo, seguidos del índice del puerto de salida y el puerto de entrada respectivo.

B.4. Desafíos para la Generación Automática

La estructura de los archivos `.pd` presenta una extrema fragilidad. El hecho de que las conexiones dependan exclusivamente del índice posicional numérico de la línea en la que se declaró el objeto significa que cualquier reordenamiento en el archivo, o la inserción de un nuevo elemento sin recalcular minuciosamente todas las conexiones de índices, corrompe completamente el programa y sus relaciones.

Además, el formato `.pd` carece de una documentación oficial estandarizada de su gramática o de algoritmos claros para las coordenadas espaciales, las cuales son requeridas para que los objetos y cables no se dibujen superpuestos de forma caótica.

Esta inherente complejidad estructural, sumada a la falta de semántica explícita (basada en nombres o UUIDs), dificulta enormemente la generación automática directa de código para Pure Data mediante inteligencia artificial. Es precisamente esta barrera la que justifica y hace indispensable el uso de metodologías dirigidas por modelos (MDA) y DSL estructurados. Al contar con un modelo formal y abstracto, se puede aislar esta complejidad y delegar a un transpilador determinista la compleja y propensa a errores tarea de generar la sintaxis `.pd` válida.

Índice de figuras

A.1. Representación de una onda sonora básica ilustrando amplitud y frecuencia. 14

A.2. Arquitectura en bloques de un sintetizador sustractivo clásico. 15

A.3. Curva de un generador de envolvente ADSR. 17

A.4. Representación del campo estéreo y el efecto de paneo. 19

A.5. Ejemplo de una cadena de señal (*Signal Chain*) con múltiples procesadores de efectos. 20

B.1. Interfaz visual de un *patch* en Pure Data. 24

B.2. Consola principal de Pd-l2ork destacando la casilla DSP. 26